

joBot : Job Search and Recommendation Chatbot (AI-Based System)

BY

Sandesh Upreti(79010792)

Orchid International College

A Project Report Submitted to

Pawan Niroula

Bachelor in Computer Science and
Technology

Kathmandu

March, 2025

Contents

- Abstract 5**
- Acknowledgement 6**
- 1 Introduction 7**
 - 1.1 Background 7
 - 1.2 Project Definition 7
 - 1.3 Objectives 7
 - 1.4 Scope 7
 - 1.5 Limitations 8
 - 1.6 Future Enhancements 8
- 2 System Analysis and Design 9**
 - 2.1 Diagrams 9
 - 2.2 Algorithm Details 11
 - 2.2.1 Introduction to Ollama 11
 - 2.2.2 Introduction to Sentence Transformers 11
 - 2.2.3 Introduction to BM25 11
 - 2.2.4 Key Features 12
- 3 Implementation and Testing 13**
 - 3.1 Implementation 13
 - 3.1.1 Python 13
 - 3.1.2 Flask 13
 - 3.2 Testing 14
- 4 Conclusion 15**

List of Figures

1	Level 0 DFD	9
2	Level 1 DFD	9
3	System Flowchart	10

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
BM25	Best Matching 25
DFD	Data Flow Diagram
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
API	Application Programming Interface
CSV	Comma-Separated Values
CV	Curriculum Vitae
IDF	Inverse Document Frequency

Table 1: List of Abbreviations

Abstract

In developing countries like Nepal, many graduates struggle to find jobs in their respective fields. Despite numerous opportunities, many job openings remain unnoticed by qualified candidates.

Job selection is often based on expertise, skills, availability, and personal preferences. However, a gap exists between job seekers and suitable opportunities, making it difficult for qualified candidates to find relevant roles.

This project aims to bridge that gap by connecting job seekers with opportunities that match their qualifications and preferences. By making relevant job openings easily accessible, this platform enhances employment prospects and empowers individuals to make informed career choices.

joBot is an AI-powered job search and recommendation chatbot system designed to assist job seekers. Leveraging Natural Language Processing (NLP) and Machine Learning (ML), joBot offers personalized job recommendations based on an individual's skills, preferences, and career goals.

Key Features:

- Search for jobs based on specific keywords and criteria.
- Receive personalized career recommendations.
- Get real-time updates on relevant job opportunities.

Acknowledgement

The completion of this project would not have been possible without the guidance and support of several individuals. I would like to express my sincere gratitude to:

- **Mr. Pawan Niroula**, my project supervisor, for his invaluable guidance and constant encouragement.
- **Orchid International College**, for providing the resources and infrastructure necessary for the project.

I would also like to thank my friends for their unwavering support and the online community for open-source libraries that helped shape this project.

1 Introduction

1.1 Background

Many graduates in Nepal struggle to find jobs in their respective fields. Despite the availability of opportunities, many job openings go unnoticed. Job selection is usually based on expertise, schedule, and skills. The goal of this project is to provide job opportunities to users based on their feasibility and availability.

1.2 Project Definition

The Job Search & Recommendation System is an AI-powered interactive chatbot that automates job searches and provides relevant recommendations. The system ensures that users receive only the necessary information without overwhelming them with excessive details.

1.3 Objectives

The main objectives of this system are:

- Provide job recommendations based on user preferences.
- Highlight unexplored and newly updated job opportunities.
- Offer a user-friendly interface for job exploration and reminders.

1.4 Scope

The scope of the system includes:

- **Job Search Functionality:** Search based on location, expertise, salary, etc.
- **Easy Interaction:** Users can upload CVs for automated job matching.
- **Recommendation System:** AI-driven recommendations based on job trends and user data.
- **Reminders:** Notifications about hiring deadlines and job expiry dates.

1.5 Limitations

The system has the following limitations:

- **Search Domain:** Requires investment to extract job data from international sources.
- **Time Consumption:** Data fetching and processing may be slow due to hardware and software limitations.
- **Internet Dependency:** Relies on web-based job listings.
- **Maintenance:** Requires frequent updates to stay relevant.
- **Accuracy:** The AI model continuously learns but may not always provide perfect matches.

1.6 Future Enhancements

- Implement CV parsing to extract skills and recommend jobs accordingly.
- Expand search capabilities beyond predefined domains.

2 System Analysis and Design

2.1 Diagrams

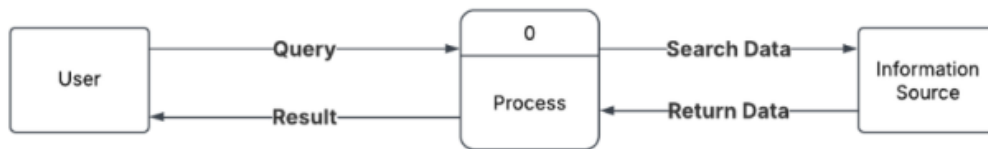


Figure 1: Level 0 DFD

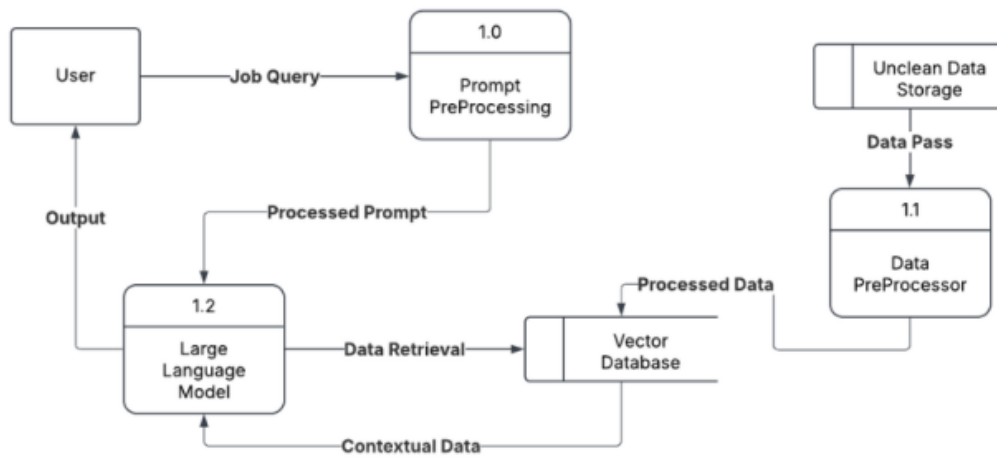


Figure 2: Level 1 DFD

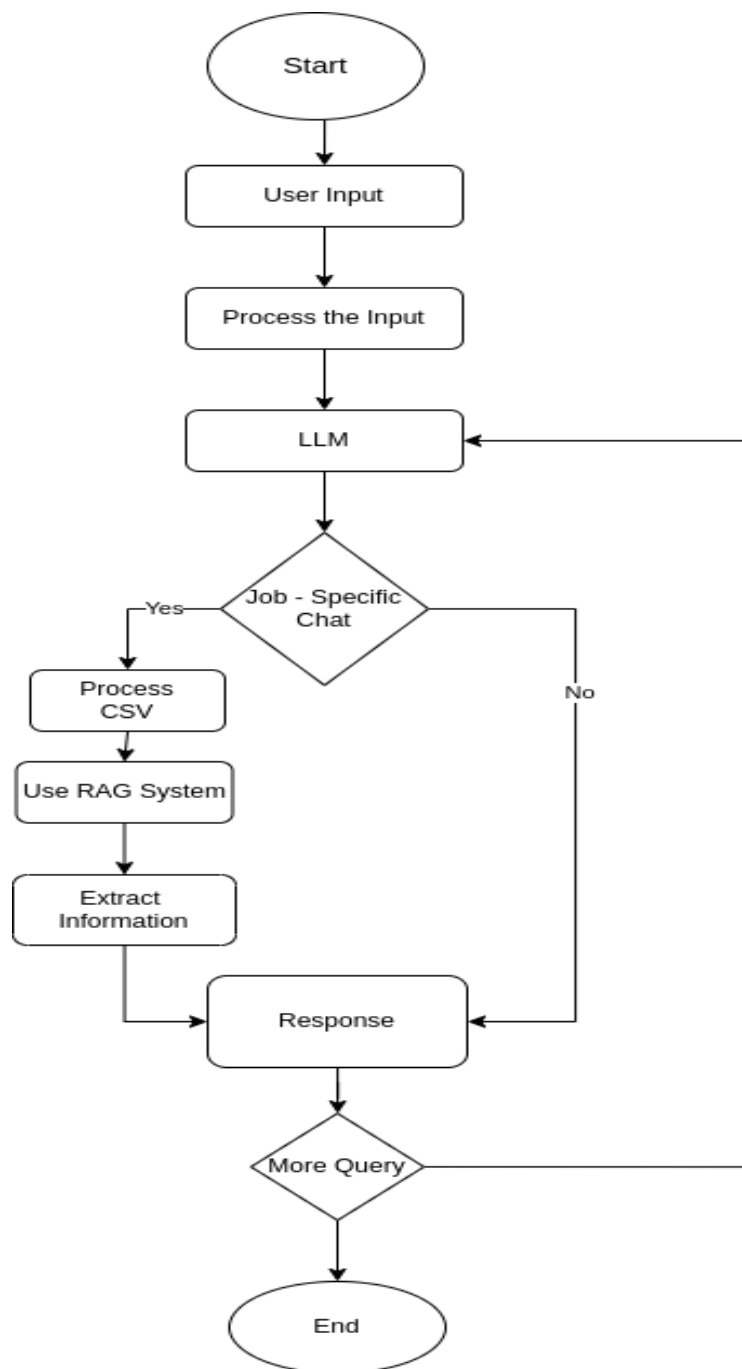


Figure 3: System Flowchart

2.2 Algorithm Details

2.2.1 Introduction to Ollama

Ollama is an AI platform that enables developers to integrate large language models (LLMs) into applications. It simplifies AI deployment and management, allowing for seamless integration with various workflows.

2.2.2 Introduction to Sentence Transformers

Sentence Transformers is a Python library built on top of the Hugging Face Transformers and PyTorch/TensorFlow, designed for efficient sentence embeddings. It enables semantic search, clustering, paraphrase detection, information retrieval, and more by converting sentences into dense vector representations.

The mathematical representation for Sentence Transformers typically involves a neural network model that learns embeddings for sentences. The goal is to minimize the distance between semantically similar sentences and maximize the distance between unrelated ones. This is typically done using a cosine similarity measure or Euclidean distance in the embedded vector space.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where A and B are sentence embeddings.

2.2.3 Introduction to BM25

BM25 (Best Matching 25) is a ranking function used in information retrieval to find the most relevant documents based on a query. It improves upon TF-IDF by considering term frequency saturation and document length normalization.

The BM25 formula is defined as:

$$\text{BM25}(q, D) = \sum_{i=1}^n \text{IDF}(t_i) \cdot \frac{f(t_i, D) \cdot (k_1 + 1)}{f(t_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgDL}}\right)}$$

Where: - $f(t_i, D)$ is the frequency of term t_i in document D , - $|D|$ is the length of the document, - avgDL is the average document length in the collection, - k_1 and b are tuning parameters, and - $\text{IDF}(t_i)$ is the inverse document frequency for term t_i .

BM25 improves on the TF-IDF formula by handling document length normalization and diminishing returns for term frequency, making it more robust in real-world scenarios.

2.2.4 Key Features

- **Model Access:** Supports advanced LLMs for tasks such as chatbot interactions and automated content generation.
- **User-Friendly Interface:** Simplifies AI integration for developers.
- **Offline Usage:** Allows local execution of models, reducing internet dependency.

3 Implementation and Testing

3.1 Implementation

The implementation of this project is carried out using Python and Flask. These technologies provide a robust and scalable foundation for the chatbot system.

3.1.1 Python

Python is a versatile, high-level programming language widely used for web development, data science, artificial intelligence, and automation. Its simplicity and extensive library support make it an ideal choice for implementing the job recommendation system.

Key Features of Python:

- Easy-to-read and maintainable syntax.
- Extensive libraries such as Pandas, NumPy, and Scikit-learn for data processing.
- Strong support for integration with databases and APIs.
- High compatibility with AI and machine learning frameworks like TensorFlow and PyTorch.

3.1.2 Flask

Flask is a lightweight, micro web framework for Python. It is designed to create web applications with minimal overhead, making it perfect for developing a chatbot API.

Key Features of Flask:

- Simple and flexible architecture for web development.
- Built-in support for routing and middleware integration.
- Compatibility with Jinja2 templating for dynamic web pages.
- Seamless integration with databases like PostgreSQL and SQLite.

3.2 Testing

Testing is a crucial phase to ensure that the system functions as expected. Various test cases were executed to validate different functionalities.

Testing Table:

Test ID	Test Case	Input	Expected Output
T1	Job Search by Keyword	Python Developer	List of relevant job postings
T2	Job Search by Location	Kathmandu	Jobs available in Kathmandu
T3	Recommendation System	User skills: Python, AI	Recommended AI-related jobs
T5	Response Time	Query execution	Response within 4 minutes

Table 2: Test Table

4 Conclusion

The implementation of this project successfully integrates AI-based job recommendations using Python and Flask. Extensive testing confirms the effectiveness of the chatbot system in providing relevant job suggestions. Future enhancements will focus on improving response accuracy and expanding job search capabilities.

References

1. Hugging Face - Sentence Transformers Documentation.
<https://www.sbert.net>
2. Ollama AI Platform.
<https://ollama.com>
3. Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
Foundations and Trends in Information Retrieval.
4. Flask Documentation.
<https://flask.palletsprojects.com>
5. Python Software Foundation - Python Programming Language.
<https://www.python.org>
6. ChromaDB Documentation.
<https://docs.trychroma.com>
7. Bird, S., Loper, E., & Klein, E. (2009). *Natural Language Processing with Python*.
O'Reilly Media.